

CocinaP

Commercial & Legal Vision

Database, scalability, laws and regulation
July 2026

1. Introduction

CocinaP currently runs as a local system with no external database: configuration is stored in a flat JSON file and alarms live in volatile memory. This architecture is sufficient for a single user, but does not scale to a commercial product with multiple users, persistent history, analytics, or cloud operation.

This document explores the options for evolving CocinaP into a commercial, public-facing product, covering databases, cloud infrastructure, business models, and applicable legal and regulatory obligations.

2. Database

2.1 Current State

- Configuration: Flat JSON file (cocinap_config.json). Read/written with json.load/json.dump.
- Alarms: In-memory list (max 50 events). Lost when the server shuts down.
- Users: No user concept exists. One local server per installation.
- FCM Tokens: JSON list, not tied to users.

2.2 Database Options

For a commercial product, real persistence is needed. Options are divided into local and cloud:

OPTION A - SQLite (Embedded local database)

- Pros: No external server required. Single file, easy to backup. Zero configuration. Sufficient for local use with persistent history.
- Cons: Does not handle heavy concurrency. Not scalable to multi-device real-time. No direct remote access.
- Recommended use: Store alarms, per-user local config, event logs.
- Python library: sqlite3 (built-in). Flutter: sqflite.

OPTION B - PostgreSQL / MySQL (Local or dedicated server)

- Pros: Real concurrency, complex queries, referential integrity, remote access.
- Cons: Requires installation and maintenance of a database server. More complex than SQLite.
- Recommended use: Multiple local users or on a local network with advanced query needs.
- Python library: psycopg2 / mysql-connector-python. Flutter: postgresql-dart / mysql1.

OPTION C - Firebase Firestore (Cloud database)

- Pros: Real-time sync between devices. Access from anywhere. Automatic scaling. Native integration with FCM and Firebase Auth.
- Cons: Requires Internet connection. Recurring costs based on usage. Data outside the local network.
- Recommended use: Push notifications, cloud alarms, remote access from mobile app, multi-device.
- Flutter library: cloud_firestore. Python: firebase-admin (already included).

OPTION D - MongoDB (NoSQL database)

- Pros: Flexible schema, easy to scale, good for unstructured data like event logs.
- Cons: Higher resource consumption. Learning curve for queries.
- Recommended use: Event log storage and detection session records.
- Python library: pymongo. Flutter: mongo_dart.

OPTION E - SQLite local + Firebase Cloud (Hybrid)

- SQLite for local data: alarms, config, logs (works offline).
- Firebase Firestore for cloud sync: share alarms across devices, remote access, cloud backup.
- Firebase Auth for user management.
- Pro: System works offline and syncs when connectivity is available.
- Con: Higher implementation complexity (two databases to keep in sync).

3. User Management and Accounts

3.1 Current Model

- No authentication. Any device on the local network can connect to the server.
- Suitable for a single-family home. Insufficient for a commercial product.

3.2 Proposed Roles

- Local Administrator: Owner of the Windows server. Full access to configuration and alarms.

- Mobile User: Household members. View video and alarms, receive notifications. No access to critical config.
- Guest: Temporary access to live video only (no history or configuration).
- Cloud Supervisor (future): Remote access via Internet authenticated with Firebase Auth.

3.3 Authentication Options

- Firebase Auth: Email + password, Google Sign-In, Apple Sign-In (iOS). Direct integration with Firestore.
- Local JWT: The Windows server generates JWT tokens for mobile devices on the local network. Simple with no external dependency.
- PIN Authentication: Numeric code displayed on the server GUI, entered in the mobile app to link.

4. Commercialization and Business Model

Different possible models for bringing CocinaP to market:

MODEL A - One-time payment (Perpetual license)

- The user buys the software once and uses it forever.
- Pros: Simple, consumer-friendly, no recurring fees.
- Cons: One-time revenue per customer. No recurring income to sustain ongoing development.

MODEL B - Monthly/Annual subscription

- The user pays a recurring fee for using the software.
- Pros: Predictable, recurring revenue. Funds cloud infrastructure and support.
- Cons: Entry barrier (subscription fatigue). Requires maintaining cloud infrastructure.

MODEL C - Freemium

- Limited free version (1 camera, 1 mobile, no cloud) and paid version with advanced features (multi-camera, multi-mobile, cloud backup, unlimited history, analytics).
- Pros: Large user base, natural conversion to paid.
- Cons: Complexity of maintaining two versions. Server costs for free users.

MODEL D - Hardware + Software (KIT)

- Sell a physical kit including a Raspberry Pi + camera + preinstalled software license.
- Pros: Turnkey product. Higher margin on hardware. Controlled experience.
- Cons: Inventory and shipping logistics. Hardware support. Higher upfront investment.

5. Cloud Infrastructure

5.1 Possible Cloud Components

- Database: Firebase Firestore or Supabase (PostgreSQL) for alarms, cloud config, and user data.
- Authentication: Firebase Auth or Supabase Auth.
- Push notifications: Firebase Cloud Messaging (already implemented) + APNs for iOS.
- File storage: Firebase Storage or AWS S3 for alarm screenshot storage.
- API gateway: If remote access without depending on the local server is desired, an intermediary cloud backend.

5.2 Full Cloud vs. Hybrid

CocinaP cannot be 100% cloud because video detection requires the local camera. AI processing must happen on the user's computer. The cloud complements by:

- Storing and syncing alarms across devices.
- Enabling remote access when the user is away from home.
- Providing aggregated analytics (optional, with consent).
- Acting as a relay for push notifications when the local server has no Internet (via FCM).

5.3 Cloud Providers

- Firebase (Google): Most complete for mobile. Auth, Firestore, FCM, Storage, Crashlytics.
- Supabase: Open source, PostgreSQL, Auth, Storage.
- AWS / Azure / GCP: Enterprise solutions for when the product scales significantly.

6. Laws and Regulations

6.1 Data Protection

When becoming a commercial product handling user data, CocinaP must comply with data protection laws depending on the user's country:

- GDPR (General Data Protection Regulation) - Europe:
 - Applies if there are users in the European Union.
 - Requires explicit consent to collect data.
 - Right to erasure: users can request deletion of their data.
 - Data breach notification within 72 hours.
 - Appoint a DPO (Data Protection Officer) if processing data at scale.
- LGPD (Lei Geral de Protecao de Dados) - Brazil:
 - Very similar to GDPR. Applies if there are users in Brazil.
 - Explicit consent, right of access, rectification and deletion.
- LFPDPPP (Federal Law on Protection of Personal Data) - Mexico:
 - Applies to personal data held by private parties.
 - Requires privacy notice and consent.
- CCPA (California Consumer Privacy Act) - California, USA:
 - Right to know what data is collected.
 - Right to delete personal data.
 - Right to opt-out of data selling.

6.2 Product Safety Laws

As a security system (fire detection), CocinaP may be subject to product safety regulations:

- ISO 7240 - Fire detection and alarm systems (international standard).
- EN 14604 - Smoke detectors for the European market.
- UL 217 - Smoke detectors for the United States.
- Although CocinaP does not replace a certified smoke detector, if marketed as a security system, it must clarify it is an assistance tool, not a replacement for certified systems.
- Include clear disclaimers in documentation and the app.

6.3 E-Commerce Laws

- Digital Services Act (DSA) - Europe: Transparency for digital platforms.
- COPPA - USA: If the product can be used by children under 13, parental consent is required.
- Consumer Protection Law - Each country: Right of withdrawal, warranties, clear information.

6.4 Intellectual Property

- Register the CocinaP trademark in each country where it is sold.
- Protect the source code with a commercial license (not open source).
- The YOLO11n model is open source (AGPL-3.0). Verify compatibility with the business model. Ultralytics offers commercial licenses.
- Consider patents if there is unique innovation in the detection or analysis method.

7. Privacy and Ethics

7.1 Key Considerations

- CocinaP uses a video camera in the home. This carries important ethical responsibilities:
- Never transmit video to external servers without explicit user consent.
- All video processing must remain local (as it currently does).
- If cloud alarm backup is implemented, only send metadata (type, timestamp, severity), not images, unless the user explicitly consents.
- Alarm screenshots must be obfuscated or stored only locally.

7.2 Required Legal Documents

- Privacy Policy: What data is collected, how it is used, who it is shared with, user rights.
- Terms of Service: User responsibilities, provider liability limitations, usage license.
- Camera Consent Notice: Inform the user that the application uses the camera.
- Disclaimer: CocinaP is an assistance tool. It does NOT replace certified smoke detectors or professional security systems. The user is solely responsible for their home safety.

7.3 Implementation Recommendations

- Add a consent screen on the first app launch (first-launch dialog).
- Store user consent in the local configuration.
- Allow the user to export and delete all their data.
- Do not collect telemetry data without consent.
- If analytics are implemented, make them anonymous and aggregated.